

# Backpack Hero

Documentation Technique

---

26 novembre 2025

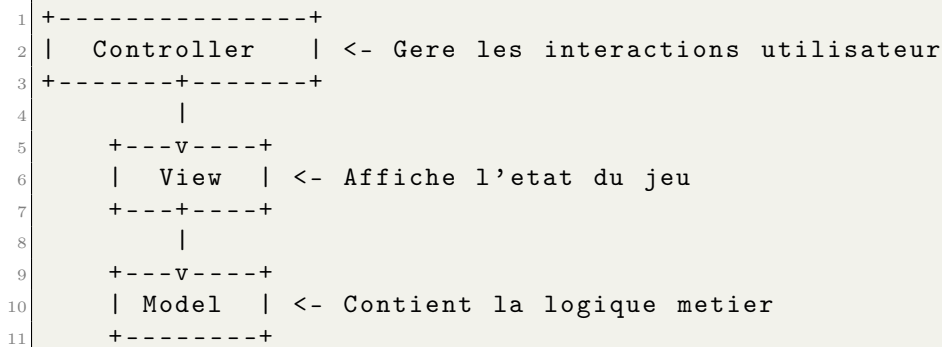
# Table des matières

|          |                                                |           |
|----------|------------------------------------------------|-----------|
| <b>1</b> | <b>Architecture générale</b>                   | <b>3</b>  |
| 1.1      | Vue d'ensemble . . . . .                       | 3         |
| <b>2</b> | <b>Architecture détaillée</b>                  | <b>3</b>  |
| 2.1      | Modèle (Model) . . . . .                       | 3         |
| 2.1.1    | Classe Hero . . . . .                          | 3         |
| 2.1.2    | Classe Backpack . . . . .                      | 3         |
| 2.1.3    | Hierarchie Equipment . . . . .                 | 4         |
| 2.1.4    | Système de combat . . . . .                    | 4         |
| 2.1.5    | Système de donjon . . . . .                    | 5         |
| 2.1.6    | Classe GameData . . . . .                      | 6         |
| 2.2      | Vue (View) . . . . .                           | 6         |
| 2.3      | Contrôleur (Controller) . . . . .              | 6         |
| <b>3</b> | <b>Principes de conception appliqués</b>       | <b>7</b>  |
| 3.1      | SOLID . . . . .                                | 7         |
| 3.1.1    | Single Responsibility Principle . . . . .      | 7         |
| 3.1.2    | Open/Closed Principle . . . . .                | 7         |
| 3.1.3    | Liskov Substitution Principle . . . . .        | 7         |
| 3.1.4    | Interface Segregation Principle . . . . .      | 7         |
| 3.1.5    | Dependency Inversion Principle . . . . .       | 7         |
| 3.2      | Patterns de conception . . . . .               | 7         |
| 3.2.1    | Pattern MVC (Model-View-Controller) . . . . .  | 7         |
| 3.2.2    | Pattern Strategy . . . . .                     | 8         |
| 3.2.3    | Pattern Template Method . . . . .              | 8         |
| 3.2.4    | Pattern Value Object . . . . .                 | 8         |
| 3.3      | Bonnes pratiques Java . . . . .                | 8         |
| 3.3.1    | Encapsulation . . . . .                        | 8         |
| 3.3.2    | Validation des arguments . . . . .             | 8         |
| 3.3.3    | Immuabilité quand possible . . . . .           | 8         |
| 3.3.4    | Defensive copying . . . . .                    | 8         |
| <b>4</b> | <b>Extensibilité pour les phases suivantes</b> | <b>8</b>  |
| 4.1      | Phase 2 : Interactions et effets . . . . .     | 8         |
| 4.2      | Phase 3 : Génération procédurale . . . . .     | 9         |
| 4.3      | Avantages de l'architecture actuelle . . . . . | 9         |
| <b>5</b> | <b>Diagramme de classes (simplifié)</b>        | <b>9</b>  |
| <b>6</b> | <b>Conclusion</b>                              | <b>11</b> |

# 1 Architecture générale

Le projet suit le pattern architectural **MVC (Modèle-Vue-Contrôleur)** pour séparer clairement les responsabilités.

## 1.1 Vue d'ensemble



Architecture MVC

# 2 Architecture détaillée

## 2.1 Modèle (Model)

Le modèle contient toute la logique métier du jeu.

### 2.1.1 Classe Hero

Représente le héros avec ses attributs :

- Points de vie (actuels et max)
- Énergie (actuels et max)
- Mana
- Or
- Expérience et niveau
- Protection (temporaire, pendant le combat)
- Sac à dos (backpack)

**Responsabilités :**

- Gérer les points de vie
- Gérer l'énergie, le mana et l'or
- Subir/infliger des dégâts
- Gagner de l'expérience

### 2.1.2 Classe Backpack

Gère le sac à dos du héros comme une grille 5×7.

**Responsabilités :**

- Placer des équipements (avec rotation)

- Vérifier si un équipement peut être placé
- Retirer des équipements
- Trouver un équipement à une position donnée

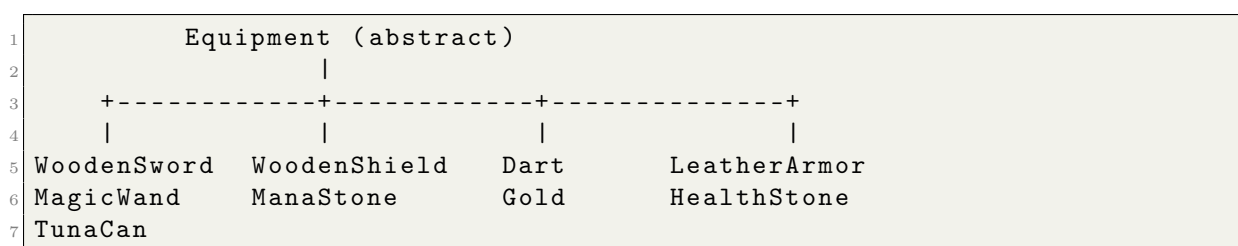
#### Structure interne :

- `boolean[][] cells` : Grille des cases disponibles
- `List<PlacedEquipment>` : Liste des équipements placés

#### Classe interne `PlacedEquipment` :

- `Equipment equipment` : L'équipement
- `int row, col` : Position
- `int rotation` : Rotation (0°, 90°, 180°, 270°)

### 2.1.3 Hiérarchie Equipment



Hiérarchie des équipements

#### Classe abstraite `Equipment` :

- Attributs : nom, forme, coûts (énergie, mana, or), rareté, prix
- Méthode abstraite : `use(CombatContext context)`
- Méthode : `getRotatedShape(int rotation)` pour la rotation

#### Exemples d'équipements :

- **WoodenSword** : Inflige 6 dégâts, coûte 1 énergie, forme 1×3
- **WoodenShield** : Ajoute 7 protection, coûte 1 énergie, forme 2×2
- **MagicWand** : Inflige 8 dégâts, coûte 1 mana, forme 1×2
- **ManaStone** : Fournit 1 mana au début du combat, forme 1×1
- **HealthStone** : Soigne 1 PV, utilisable 1 fois par tour, forme 1×1

### 2.1.4 Système de combat

**Classe `Combat`** : Gère un combat au tour par tour.

#### Responsabilités :

- Alternier entre tour du héros et tour des ennemis
- Gérer l'utilisation des équipements
- Vérifier les conditions de victoire/défaite
- Distribuer expérience et récompenses

#### Flux du combat :

1. Initialisation : Compter mana stones, planifier actions ennemies
2. Tour du héros : Utiliser équipements, dépenser énergie/mana
3. Fin du tour du héros : Exécuter actions ennemies
4. Répéter jusqu'à victoire ou défaite

#### Classe abstraite `Enemy` :

- Attributs : nom, PV, protection, récompense XP
- Méthode abstraite : `chooseAction()` → ATTACK ou DEFEND
- Méthodes : `executeAction()`, `takeDamage()`, `addProtection()`

#### Ennemis implémentés :

- **SmallRatWolf** : 15 PV, 8 dégâts, 5 protection, 4 XP
- **RatWolf** : 20 PV, 12 dégâts, 7 protection, 6 XP

**Classe CombatContext** : Fournit le contexte aux équipements durant le combat :

- Référence au héros
- Liste des ennemis
- Ennemi ciblé

### 2.1.5 Système de donjon



Structure du donjon

#### Classe Dungeon :

- Contient plusieurs étages (Floor)
- Garde une référence au héros
- Gère la progression entre étages

#### Classe Floor :

- Grille 5×11 de salles (Room)
- Gère la salle actuelle
- Vérifie les chemins entre salles (BFS)

#### Classe Room :

- Type : CORRIDOR, ENEMY, TREASURE, MERCHANT, HEALER, EXIT
- Position (row, col)
- Contenu (RoomContent)
- État visité

#### Classe RoomContent :

- Liste d'ennemis
- Liste d'équipements
- Paramètres de guérison (coût, montant)

**Classe DungeonGenerator** : Pour la Phase 1, génère 3 étages codés en dur avec :

- 3 salles d'ennemis par étage
- 1 marchand par étage
- 1 guérisseur par étage
- 2 trésors par étage
- 1 sortie par étage

### 2.1.6 Classe GameData

**Point central du modèle** qui coordonne tous les éléments.

**Responsabilités :**

- Créer et initialiser le jeu
- Gérer l'état du jeu (EXPLORING, COMBAT, TREASURE, MERCHANT, HEALER, VICTORY, DEFEAT)
- Gérer les déplacements dans le donjon
- Initier les combats
- Gérer les interactions avec marchands/guérisseurs/trésors

**États du jeu (GameState enum) :**

- EXPLORING : Exploration du donjon
- COMBAT : En combat
- TREASURE : Dans une salle de trésor
- MERCHANT : Chez un marchand
- HEALER : Chez un guérisseur
- VICTORY : Victoire (sortie du dernier étage)
- DEFEAT : Défaite (héros mort)

## 2.2 Vue (View)

**Interface GameView :**

```
1 void display(GameData gameData)
2 int getWidth()
3 int getHeight()
```

**Implémentation ConsoleView (Phase 1) :**

- Affiche l'état du jeu en mode texte
- Affiche la carte du donjon
- Affiche l'état du combat
- Affiche le contenu des salles spéciales

**Future implémentation avec Zen (Phase 2+) :**

- Interface graphique complète
- Affichage visuel du sac à dos
- Animations de combat
- Carte interactive

## 2.3 Contrôleur (Controller)

**Classe GameController :**

**Responsabilités :**

- Gérer la boucle de jeu
- Lire les entrées utilisateur
- Appeler les méthodes appropriées du modèle
- Demander l'affichage à la vue

**Flux principal :**

```
1 start()
2 |
3 +-> while (running && hero.isAlive())
4     +-> view.display(gameData)
5     +-> handleInput()
6         | +-> handleExploringInput()
7         | +-> handleCombatInput()
8         | +-> handleTreasureInput()
9         | +-> handleMerchantInput()
10        | +-> handleHealerInput()
11    +-> loop
```

Flux du contrôleur

## 3 Principes de conception appliqués

### 3.1 SOLID

#### 3.1.1 Single Responsibility Principle

Chaque classe a une responsabilité unique :

- **Hero** : Gérer l'état du héros
- **Backpack** : Gérer le sac à dos
- **Combat** : Gérer un combat
- **Floor** : Gérer un étage du donjon

#### 3.1.2 Open/Closed Principle

- Classes abstraites **Equipment** et **Enemy** ouvertes à l'extension
- Ajout de nouveaux équipements sans modifier le code existant
- Ajout de nouveaux ennemis sans modifier le code existant

#### 3.1.3 Liskov Substitution Principle

- Tout **Enemy** peut être utilisé de manière interchangeable
- Tout **Equipment** peut être utilisé de manière interchangeable

#### 3.1.4 Interface Segregation Principle

- Interface **GameView** minimale et ciblée

#### 3.1.5 Dependency Inversion Principle

- **GameController** dépend de l'interface **GameView**, pas d'une implémentation concrète

## 3.2 Patterns de conception

### 3.2.1 Pattern MVC (Model-View-Controller)

Séparation claire entre logique métier, présentation et contrôle.

### 3.2.2 Pattern Strategy

Les ennemis choisissent leur action via `chooseAction()` (méthode polymorphe).

### 3.2.3 Pattern Template Method

`Equipment.use(CombatContext)` est implémentée différemment par chaque sous-classe.

### 3.2.4 Pattern Value Object

`PlacedEquipment` est un objet immuable représentant un équipement placé.

## 3.3 Bonnes pratiques Java

### 3.3.1 Encapsulation

- Tous les champs sont `private`
- Accès via getters/setters ou méthodes métier
- Validation dans les setters et constructeurs

### 3.3.2 Validation des arguments

```
1 public Hero(String name) {  
2     this.name = Objects.requireNonNull(name);  
3     if (maxHP <= 0) {  
4         throw new IllegalArgumentException("Max HP must be positive");  
5     }  
6 }
```

### 3.3.3 Immuabilité quand possible

- Énumérations (`GameState`, `RoomType`, `Rarity`, `EnemyAction`)
- Classes value object

### 3.3.4 Defensive copying

Les getters retournent des copies des collections :

```
1 public List<Enemy> getEnemies() {  
2     return new ArrayList<>(enemies);  
3 }
```

## 4 Extensibilité pour les phases suivantes

### 4.1 Phase 2 : Interactions et effets

Ajout prévu :

- Interface `EquipmentEffect` pour les interactions entre équipements
- Classe `StatusEffect` pour les effets de combat
- Classe `Curse` (sous-classe d'`Equipment`) pour les malédictions



**Modifications nécessaires :**

- Equipment : Méthode `onCombatStart()`, `onTurnStart()`, `onTurnEnd()`
- Hero : Liste de `StatusEffect`
- Combat : Gestion des effets de combat

## 4.2 Phase 3 : Génération procédurale

**Ajout prévu :**

- Algorithme de génération de donjon (garantir connexité)
- Système de probabilités pour les drops d'équipements
- Plus d'ennemis (sorcier-grenouille, ombre vivante, reine des abeilles)

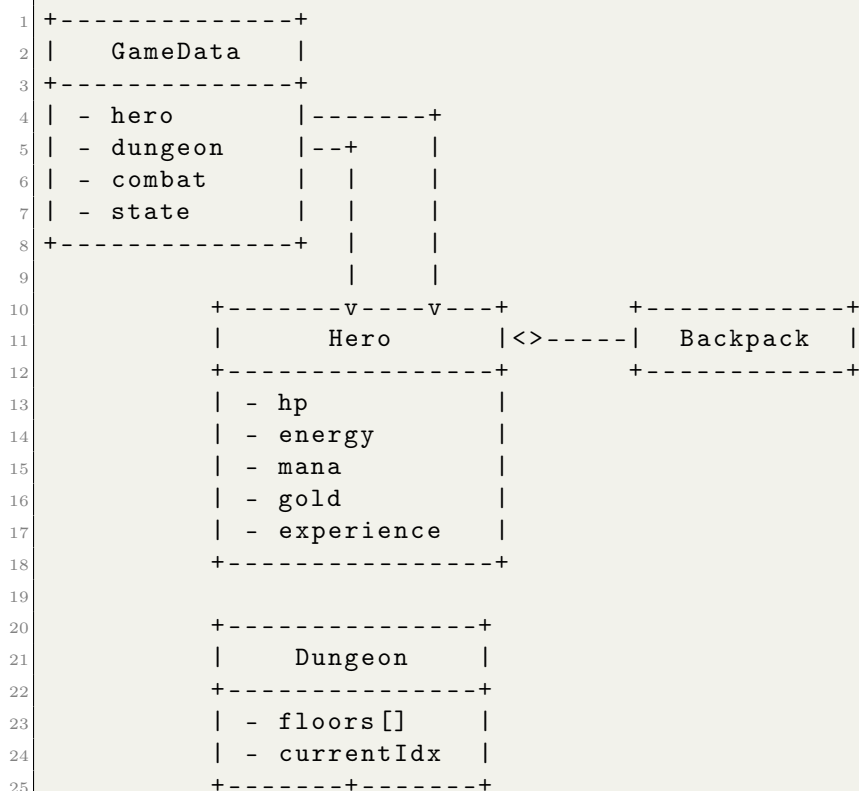
**Modifications nécessaires :**

- `DungeonGenerator` : Génération aléatoire au lieu de hard-coded
- `RoomContent` : Génération aléatoire des équipements
- Système de rareté pour les drops

## 4.3 Avantages de l'architecture actuelle

1. **Modularité** : Chaque composant peut être testé indépendamment
2. **Extensibilité** : Facile d'ajouter de nouveaux équipements/ennemis
3. **Maintenabilité** : Code organisé et documenté
4. **Réutilisabilité** : Classes génériques réutilisables
5. **Testabilité** : Séparation modèle/vue facilite les tests

## 5 Diagramme de classes (simplifié)



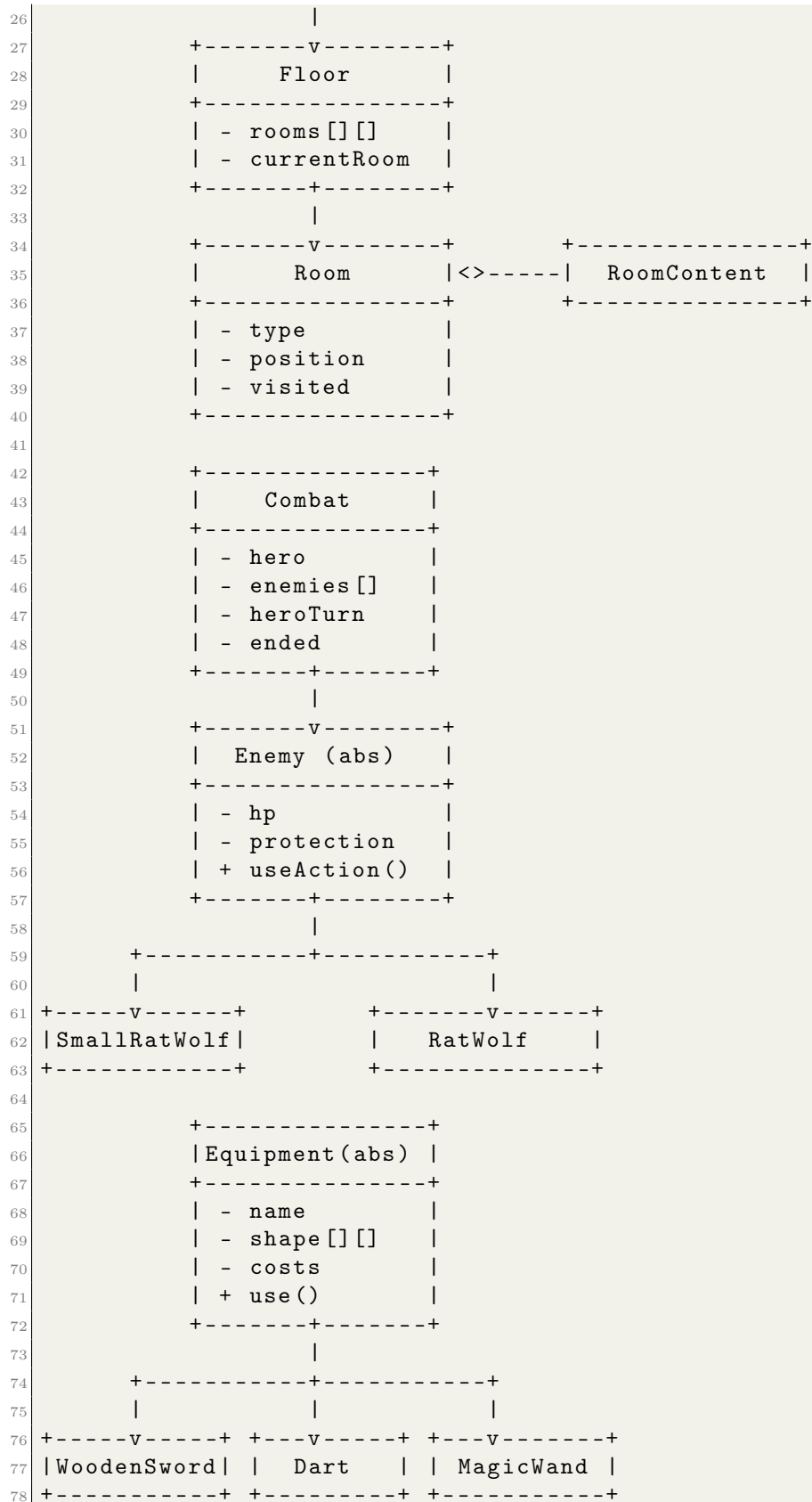


Diagramme de classes

## 6 Conclusion

L'architecture actuelle offre une base solide pour les phases suivantes. Elle respecte les principes SOLID, utilise des patterns de conception appropriés, et est conçue pour être facilement extensible.

Le code est propre, documenté (Javadoc), et suit les conventions Java. Aucune variable globale n'est utilisée, les méthodes sont courtes ( $< 20$  lignes), et tous les champs sont privés avec validation appropriée.